

AI认知诊断引擎

CDM驱动的知识状态诊断系统 · 安装手册

版本 1.0.0 | 2025年5月

⚠ 内容使用AI生成

支持 Docker Compose 容器化部署 和 手动部署 两种方式
适用于 Windows / macOS / Linux 操作系统

目录

- [第一章 系统要求](#)
- [第二章 Docker Compose 部署 \(推荐\)](#)
- [第三章 手动部署 — 后端](#)
- [第四章 手动部署 — 前端](#)
- [第五章 数据库初始化](#)
- [第六章 环境变量配置](#)
- [第七章 健康检查与验证](#)
- [第八章 常见安装问题](#)

第一章 系统要求

1.1 硬件要求

组件	最低配置	推荐配置
CPU	双核 2.0GHz	四核 3.0GHz+ (EM算法为计算密集型)
内存	4 GB	8 GB+
磁盘空间	10 GB	20 GB+
网络	无特殊要求	—

1.2 软件要求

Docker Compose 部署 (推荐)

软件	版本要求
Docker Engine	20.10+
Docker Compose	2.0+

手动部署

软件	版本要求	用途
Python	3.12+	后端运行时
Node.js	22+	前端构建工具
PostgreSQL	16+	数据库
npm / pnpm	9+ / 8+	前端包管理
Git	2.0+	版本管理 (可选)

⚠ **重要提示:** EM算法和蒙特卡洛采样需要较强的CPU算力。如果知识点数超过15个，建议CPU在四核以上以确保诊断响应速度。

第二章 Docker Compose 部署（推荐）

Docker Compose 一键部署将所有服务（PostgreSQL + 后端 + 前端）打包启动，无需手动配置依赖环境，是推荐的部署方式。

2.1 获取代码

```
cd /path/to/project/root
```

2.2 配置环境变量

确认项目根目录下的 `docker-compose.yml` 文件，默认配置可直接使用：

```
# docker-compose.yml 关键配置
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: cognitive_diagnosis
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"

  backend:
    build: ./backend
    environment:
      DATABASE_URL: postgresql://postgres:postgres@postgres:5432/cognitive_diagnosis
    ports:
      - "8003:8003"

  frontend:
    build: ./frontend
    ports:
      - "5173:80"
```

2.3 启动服务

```
# 构建镜像并启动所有服务（后台运行）
docker-compose up -d --build
```

```
# 查看服务状态
docker-compose ps

# 查看日志
docker-compose logs -f
```

2.4 停止服务

```
# 停止所有服务
docker-compose down

# 停止并删除数据卷（⚠ 会清空数据库）
docker-compose down -v
```

2.5 服务架构图

服务名	端口	容器名	依赖
postgres	5432	cognitive_diagnosis_postgres	—
backend	8003	cognitive_diagnosis_backend	postgres (健康检查)
frontend	5173 (→80)	cognitive_diagnosis_frontend	backend (健康检查)

第三章 手动部署 — 后端

3.1 环境准备

```
# 确认 Python 版本 ≥ 3.12
python --version

# 确认 PostgreSQL 已启动且可连接
psql -U postgres -c "SELECT version();"
```

3.2 创建数据库

```
psql -U postgres
CREATE DATABASE cognitive_diagnosis;
\q
```

3.3 安装依赖

```
cd backend

# 创建虚拟环境（推荐）
python -m venv venv
# Windows
venv\Scripts\activate
# macOS / Linux
source venv/bin/activate

# 安装 Python 依赖
pip install -r requirements.txt
```

3.4 依赖清单

包名	版本	用途
fastapi	0.111.0	API 框架
uvicorn[standard]	0.30.1	ASGI 服务器

sqlalchemy	2.0.31	ORM 数据库层
alembic	1.13.1	数据库迁移
numpy	1.26.4	矩阵运算 (CDM核心)
scipy	1.14.0	优化算法 (EM)
pandas	2.2.2	数据处理
python-pptx	0.6.23	PPT课件解析
pydantic	2.8.2	数据校验
pydantic-settings	2.3.4	配置管理
python-multipart	0.0.9	文件上传
httpx	0.27.0	HTTP客户端 (测试)
pytest	9.0.3	测试框架
pytest-cov	7.1.0	测试覆盖率
psycopg2-binary	2.9.9	PostgreSQL驱动

3.5 启动后端

```
# 设置环境变量 (或通过 .env 文件)
export DATABASE_URL=postgresql://postgres:postgres@localhost:5432/cognitive_diagnosis
export ALLOWED_ORIGINS=http://localhost:5173,http://localhost:3000

# 初始化数据库 (首次运行)
python init_data.py

# 启动服务
uvicorn app.main:app --host 0.0.0.0 --port 8003 --reload
```

 **提示：**生产环境建议使用 `gunicorn + uvicorn workers` 代替单进程 `uvicorn`，以获得更好的并发性能。

3.6 Dockerfile 说明

后端Docker镜像采用Python 3.12-slim多阶段构建，关键步骤：

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8003
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8003"]
```

第四章 手动部署 — 前端

4.1 环境准备

```
# 确认 Node.js 版本 ≥ 22
node --version # 期望 ≥ 22.0.0
npm --version  # 期望 ≥ 9.0.0
```

4.2 安装依赖

```
cd frontend

# 安装所有 npm 依赖
npm install
```

4.3 依赖清单

包名	用途
react 18 + react-dom	前端框架
antd 5	UI组件库
@ant-design/icons	图标库
@ant-design/charts	图表库（趋势图、热力图等）
zustand	轻量级状态管理
react-router-dom 6	客户端路由
axios	HTTP 请求
dayjs	日期处理
@antv/g6	知识图谱可视化
vitest + @testing-library/react	测试框架（开发）

4.4 开发模式启动

```
# 确保前端根目录存在 .env 文件
echo "VITE_API_BASE_URL=http://localhost:8003" > .env

# 启动开发服务器（热重载）
npm run dev
```

开发服务器默认监听 `http://localhost:5173`。

4.5 生产构建

```
# 构建生产版本
npm run build

# 构建产物位于 dist/ 目录
# 可用 nginx 或其他静态文件服务器托管
```

4.6 Nginx 配置

前端Docker镜像使用Node 22构建 + Nginx托管的双阶段构建。nginx.conf配置：

```
server {
    listen 80;
    server_name localhost;

    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html; # SPA 路由回退
    }

    location /api/ {
        proxy_pass http://backend:8003/; # API 反向代理
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

第五章 数据库初始化

5.1 自动初始化

项目提供了 `backend/init_data.py` 脚本，在首次启动后自动完成以下操作：

- 创建所有数据库表（基于 SQLAlchemy ORM 模型）
- 导入课程标准数据（从 `backend/data/curriculum/` 目录）
- 创建示例项目和数据（可选，通过参数控制）

5.2 手动执行初始化

```
cd backend
python init_data.py
```

5.3 数据库表结构概览

表名	说明	关键字段
projects	项目表	id, name, subject, grade, status
coursewares	课件表	id, project_id, filename, parse_status
knowledge_points	知识点表	id, code, name, level, parent_id, subject
kp_prerequisites	前驱关系表	kp_id, prerequisite_id (多对多关联)
homeworks	作业表	id, project_id, title, class_id
homework_items	题目表	id, homework_id, index, type, max_score
q_matrix	Q矩阵表	id, item_id, kp_id, value(0/1), confirmed
answer_records	答题记录表	id, item_id, student_id, answer, score, is_correct
cdm_sessions	诊断会话表	id, homework_id, model_type, status, AIC, BIC
cdm_results	诊断结果表	id, session_id, student_id, kp_id, P_mastery
root_causes	根因追溯表	id, session_id, student_id, kp_id, root_cause_id, depth

learning_paths	学习路径表	id, session_id, student_id, path_json
agent_events	Agent事件日志	id, agent_name, event_type, payload, timestamp

第六章 环境变量配置

6.1 后端环境变量

变量	默认值	说明
DATABASE_URL	sqlite:///./data/app.db	数据库连接串。Docker部署时使用PostgreSQL；本地开发可用SQLite
ALLOWED_ORIGINS	http://localhost:5173	CORS允许的前端来源，逗号分隔
HOST	0.0.0.0	服务绑定地址
PORT	8003	服务端口

6.2 前端环境变量

变量	默认值	说明
VITE_API_BASE_URL	http://localhost:8003	后端API基地址

⚠ 注意：前端环境变量以 `VITE_` 前缀命名，这是 Vite 构建工具的要求。修改后需重启前端开发服务器。

第七章 健康检查与验证

7.1 后端健康检查

```
# 方式1: 浏览器访问
http://localhost:8003/health

# 方式2: 命令行
curl http://localhost:8003/health
# 预期输出: {"status": "healthy", "timestamp": "..."}

# 方式3: 运行测试
cd backend
python -m pytest --tb=short -q -m "not slow"
```

7.2 前端验证

```
# TypeScript 类型检查
cd frontend
npx tsc --noEmit
# 预期输出: (无输出 = 0错误)

# 运行组件测试
npx vitest run --reporter=verbose
# 预期: 4个测试文件 24个测试用例 全部通过
```

7.3 完整验证清单

#	验证项	预期结果	状态
1	PostgreSQL可连接	pg_isready 返回成功	☐
2	后端 /health 端点	HTTP 200 + healthy	☐
3	前端页面可访问	访问 localhost:5173 显示仪表盘	☐
4	API 代理正常	前端可正常请求后端数据	☐
5	数据库已初始化	数据库中存在 projects 表	☐

6	TS编译通过	npx tsc --noEmit 无错误	☐
7	后端测试通过	pytest 54+ passed	☐

第八章 常见安装问题

Q1: Docker启动失败, 提示端口被占用

修改 `docker-compose.yml` 中的端口映射:

```
ports:
  - "5800:80"    # 将前端端口改为 5800
  - "18003:8003" # 将后端端口改为 18003
```

Q2: 后端启动时"数据库连接失败"

检查以下项:

- PostgreSQL服务是否已启动 (`pg_isready`)
- `DATABASE_URL` 环境变量是否正确 (主机名、端口、用户名、密码)
- 数据库 `cognitive_diagnosis` 是否已创建

Q3: 前端访问API报 CORS 错误

检查后端环境变量 `ALLOWED_ORIGINS` 是否包含前端的实际地址。例如:

```
# 如果前端在 http://localhost:3000
export ALLOWED_ORIGINS=http://localhost:3000,http://localhost:5173
```

Q4: pip install 时报 psycopg2 编译错误 (Windows)

使用二进制预编译版本:

```
pip install psycopg2-binary
```

或者将数据库切换为本地SQLite (修改 `DATABASE_URL` 为 `sqlite:///./data/app.db`)。

Q5: npm install 超时或失败

```
# 使用国内镜像源加速
npm config set registry https://registry.npmmirror.com
npm install

# 或使用 pnpm (更快)
npm install -g pnpm
pnpm install
```

Q6: 如何重置数据库?

```
# Docker 环境
docker-compose down -v
docker-compose up -d --build

# 手动环境 (PostgreSQL)
psql -U postgres -c "DROP DATABASE cognitive_diagnosis;"
psql -U postgres -c "CREATE DATABASE cognitive_diagnosis;"
cd backend && python init_data.py
```

Q7: 内存不足导致EM算法卡死

当同时诊断的知识点超过15个时，EM算法的状态空间呈指数级增长 (2^K)。系统在 $K > 15$ 时自动降级为蒙特卡洛EM。若仍出现卡死，请减少同时诊断的知识点数量或在设置中降低阈值。